C++ Fundamentals

Lesson 1 / Input, decisions, and data

Our program for today

```
Input: 3 scores
Compute: sum and average
Decide: average at least 60?
Print: the report
```

How would you write this in Python?

One complete program

```cpp
#include <iostream>

int main() {
    std::cout << "Hello, C++!\n";
    return 0;
}
```

Execution enters main. Statements end with a semicolon.

Python and C++ notation

```cpp
// Python: score = 85
int score = 85;

// Python: if score >= 60:
if (score >= 60) {
    std::cout << "pass\n";
}
```

The type precedes the name. Braces delimit a block.

Blocks and variable scope

```cpp
int score = 85;
{
    int bonus = 5;
    score += bonus;
}
std::cout << score << '\n';
// bonus is no longer in scope here
```

Inner code can use outer names. A local name ends with its block.

Compilation and execution

```
std::cout << "Hello\n"
```

A semicolon is missing. Will this version print Hello?

The first compiler diagnostic

Read the file and line number

Inspect that line and the line before it

Fix the first cause, then compile again

std:: qualifies a name

```cpp
std::cout << "AIM\n";

using std::cout;
cout << "AIM\n";
```

Both forms name the same standard library object.

Output order

```cpp
int score = 85;
std::cout << "score=" << score << '\n';
```

What appears on screen?

Input and types

First decide what each value represents

Variable types

```
int count = 3;
double average = 82.5;
char grade = 'A';
std::string name("anon");
bool passed = true;
```

Integer / fraction / character / string / condition

Initialization and later assignment

```cpp
int count = 3;  // initialization
count = 5;      // assignment

double measured = 3.5;
int truncated = static_cast<int>(measured);
```

Initialization creates an object. Assignment changes an existing one.

An explicit conversion may still discard a fractional part.

Types determine representable values

```cpp
int whole = 85;
double fraction = 85.5;

std::cout << sizeof(whole) << '\n';
std::cout << sizeof(fraction) << '\n';
```

Integers have bounds. Floating values have limited precision.

Type sizes depend on the implementation.

Input follows extraction order

```
int a = 0, b = 0;
std::cin >> a >> b;
std::cout << a - b << '\n';
```

Input 8 4. What is printed? What if they are on separate lines?

Token extraction and line input

```
std::string first, second;
std::cin >> first >> second;
// Input: poppin party
// first = "poppin", second = "party"
```

reads whitespace–separated tokens. getline reads a line.

Failed input

```cpp
int score = 0;
if (!(std::cin >> score)) {
    std::cout << "invalid input\n";
    return 1;
}
```

Where does execution go for input abc?

Practice: read and print

Read a name and a score

Print anon:85

Sample input: anon 85

Read and print: review

```
std::string name;
int score = 0;
std::cin >> name >> score;
std::cout << name << ':' << score << '\n';
```

Check types, extraction order, and output punctuation.

Operators and decisions

Predict a result, then explain the types

The value of 3 / 2

```
std::cout << 3 / 2 << '\n';
std::cout << 3 / 2.0 << '\n';
std::cout << 3 % 2 << '\n';
```

What does each line print?

When conversion happens

```cpp
int a = 3, b = 2;

double x = a / b;
double y = static_cast<double>(a) / b;
```

x is 1, y is 1.5

The destination type does not redo the division.

Tracing three division expressions

```
3 / 2                          // 1
static_cast<double>(3) / 2      // 1.5
static_cast<double>(3 / 2)      // 1.0
```

Identify operand types, perform division, then convert the result.

Assignment and comparison

```
int x = 10;
x = 3;

std::cout << (x == 3) << '\n';
std::cout << (x != 3) << '\n';
```

= changes a value. == tests equality.

Compound conditions and short-circuiting

```
if (total != 0 && score / total > 0.6) {
    std::cout << "pass\n";
}
```

If total is 0, is the right side evaluated?

AND, OR, and NOT

```
true  && false   // false
true  || false   // true
!true            // false
!(score < 0 || score > 100)
```

The last expression means the score is in [0, 100].

Updating a variable

```
int count = 3;
++count;
count += 2;
count *= 3;
```

What is count after each line?

Math functions and result types

```cpp
#include <cmath>

std::sqrt(9.0);   // 3.0
std::abs(-3);     // 3
std::floor(2.8);  // 2.0
std::ceil(2.1);   // 3.0
```

A call produces a value. Printing it still requires output.

Mutually exclusive if branches

```
if (score >= 90) {
    std::cout << "A\n";
} else if (score >= 60) {
    std::cout << "pass\n";
} else {
    std::cout << "retry\n";
}
```

For score=95, how many lines are printed?

Branch order and boundary cases

```
// Test cases for score:
// 59  -> retry
// 60  -> pass
// 89  -> pass
// 90  -> A
// 100 -> A
```

Testing both sides of each boundary catches more than testing only 80.

Practice: average and pass check

Read 3 integer scores

Print their average to 1 decimal place

Print pass if the average is at least 60

Average: calculation and formatting

```
double average = (a + b + c) / 3.0;
std::cout << std::fixed
          << std::setprecision(1)
          << average << '\n';
if (average >= 60) {
    std::cout << "pass\n";
}
```

Displaying 1 decimal place does not change the stored average.

Break

10 minutes

Next: strings, then a complete program

Loops and collections

Let a loop handle repeated work

The execution cycle of for

```
int sum = 0;
for (int i = 1; i <= 3; ++i) {
    sum += i;
}
```

Initialize once, test before the body, then update.

A loop state trace

```
i       condition       sum after body
1       true            1
2       true            3
3       true            6
4       false           6
```

The final test still runs, but its body does not.

Loop boundaries

```
for (int i = 0; i < 3; ++i) {
    std::cout << i << ' ' ;
}
```

Prints 0 1 2

What appears if the test becomes <= 3?

continue and break

```cpp
for (int i = 0; i < 5; ++i) {
    if (i == 1) continue;
    if (i == 4) break;
    std::cout << i << ' ' ;
}
```

Prints 0 2 3

continue skips this iteration. break exits the loop.

Array indices

```
int scores[5] = {94, 87, 50};

std::cout << scores[0] << '\n';
std::cout << scores[4] << '\n';
```

5 positions: 0, 1, 2, 3, 4

The remaining elements are initialized to 0.

Initialization affects element values

```
int a[3] = {0};         // 0, 0, 0
int b[3] = {5};       // 5, 0, 0
int c[3];           // uninitialized local ints
```

Uninitialized local elements are not valid values to read.

Assigning a std::array

```
std::array<int, 3> a = {94, 87, 50};
std::array<int, 3> b = {};
b = a;
b[0] = 100;
```

a[0] remains 94

Built–in arrays do not support b = a.

Copies hold independent elements

```
a:  94  87  50
b:  94  87  50   // after b = a

b[0] = 100

a:  94  87  50
b: 100  87  50
```

A value copy creates a separate object. Later changes do not synchronize.

Copies in a range loop

```
std::array<int, 3> scores = {60, 70, 80};

for (int score : scores) {
    score += 5;
}
```

After the loop, is scores[0] 60 or 65?

Changing elements through a reference

```
for (int& score : scores) {
    score += 5;
}
```

scores becomes 65, 75, 85

Here & gives an alias for each element.

Three range-loop forms

```
for (int value : scores)       // copy
for (int& value : scores)      // modify element
for (const int& value : scores) // read only
```

Choose the loop variable type based on whether elements should change.

Practice: count passing scores

Given {94, 87, 50, 60, 0}

Count the elements that are at least 60

Then find the average, expected 58.2

Two accumulators in the score task

```
int sum = 0, passed = 0;
for (int score : scores) {
    sum += score;
    if (score >= 60) ++passed;
}
double average =
    static_cast<double>(sum) / scores.size();
```

For this fixed input: sum=291, passed=3, average=58.2.

Loops and arrays checkpoint

Why does double x = 3 / 2 give 1?

Why might score += 5 leave an array unchanged?

Strings

Locate and edit a name in a string

Strings have indices too

```
std::string name("soyo");
std::cout << name.size() << '\n';
name[0] = 'S';
```

size is 4. The updated string is Soyo.

find returns a position

```
std::string members("soyo,anon,taki");
std::string::size_type pos = members.find("anon");
```

At which index does anon begin?

A missing match gives npos

```
std::string::size_type pos = members.find("rana");
if (pos != std::string::npos) {
    std::cout << pos << '\n';
}
```

Position 0 is a match. if(pos) is not a match test.

Three search boundary cases

```
members = "soyo,anon,taki";
find("soyo")  // 0: found at the start
find("taki")  // 10: found later
find("rana")  // npos: not found
```

Cover a match at 0, a later match, and no match.

replace shifts later positions

```
std::string::size_type pos = members.find("anon");
if (pos != std::string::npos) {
    members.replace(pos, 4, "rana");
}
```

Result: soyo,rana,taki

After a longer replacement, find later positions again.

Appending and erasing

```
members += ",tomori";
members.insert(0, "rana,");
members.erase(0, 5);
```

Who removes the separator when deleting a name?

State after each string edit

```
start:    soyo,anon,taki
append:   soyo,anon,taki,tomori
insert:   rana,soyo,anon,taki,tomori
erase:    soyo,anon,taki,tomori
```

Each edit starts from the previous result.

Practice: find and replace a member

Start with soyo,anon,taki

Replace anon with tomori if found

Keep the string unchanged if absent

Putting it together

Related data can belong to one object

A single record

```
struct BandRecord {
    std::string guitar;
    std::string vocal;
};

BandRecord band = {"mutsumi", "hatsune"};
std::cout << band.guitar << '\n';
```

A type describes the structure. An object stores one instance.

A type and two records

```
BandRecord a = {"mutsumi", "hatsune"};
BandRecord b = {"anon", "tomori"};
a.guitar = "soyo";
```

Two objects of the same type hold their own fields.

Integrated task: a score report

Read 3 scores into a std::array

Sum them in a loop and print the average to 1 decimal

Print the number of passing scores

Checking the integrated program

94 87 50: average 77.0, 2 passes

60 60 60: average 60.0, 3 passes

0 0 0: average 0.0, 0 passes

Check input, accumulators, division, then comparisons

Lesson 1 review

Where do you look first in a new program?

What do you inspect first when a result is wrong?

Next: how can a function change the caller’s data?